Veeam XFS Backup-Copy Cleanup v0.5.0 - Operator Guide

Applies to Veeam Backup & Replication v12 and v13 backup-copy files on manually administered Linux x86_64 hardened repositories using XFS. Published October 13, 2026. This is an independently developed filesystem utility, not a Veeam-supported retention operation.

IMPORTANT: WARNING: Destructive cleanup permanently unlinks eligible files. Stop backup-copy jobs or guarantee that none can start until the scan, purge, and logging finish. The script does not lock out the Veeam Data Mover.

1. Scope and prerequisites

Requirement	Verify before use
Supported platform	Linux x86_64, Python 3.8 or newer, mounted XFS backup-copy repository
Utilities	findmnt (util-linux), fuser (psmisc), readable /proc/self/mountinfo
Permissions	Root for --scan-purge or --purge; read access for a dry run
Locking	Trusted /run/lock and one active cleanup instance per repository
Reports	Private, operator-owned directory mode 0700 outside the repository; report files mode 0600
Excluded	Veeam Software/Infrastructure Appliance root-level deployment; unrelated backup types

Use the backup-copy repository root directly above its job directories, not a VM folder or a higher-level volume containing unrelated backups. Do not point the tool at / or a path with symlink components or nested mounts.

```
python3 --version
command -v findmnt
command -v fuser
test -r /proc/self/mountinfo
findmnt -n -o TARGET,FSTYPE --target '/mnt/YOUR_VOLUME/backups'
```

2. Installation and package verification

Use the matching v0.5.0 customer package, not the earlier v0.4.3 ZIP. The refreshed package is named veeam_xfs_cleanup_v0.5.0_client_with_guide.zip and contains veeam_xfs_cleanup_v0.5.0/ with the script, documentation and SHA256SUMS.txt.

```
unzip veeam_xfs_cleanup_v0.5.0_client_with_guide.zip
cd veeam_xfs_cleanup_v0.5.0
sha256sum -c SHA256SUMS.txt
sudo install -d -m 0700 /opt/veeam-xfs-cleanup
sudo install -m 0644 veeam_xfs_cleanup.py /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py
sudo install -d -m 0700 /var/log/veeam-xfs-cleanup
python3 /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py --version
```

If installing from a Git clone instead, copy veeam_xfs_cleanup.py from this repository to /opt/veeam-xfs-cleanup/. Do not rely on the customer ZIP filename when installing from

GitHub. The --version output must be veeam-xfs-cleanup 0.5.0.

3. Run modes

Interactive menu

```
sudo python3 /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py
```

Choose 1 for a dry scan, 2 for a fresh scan followed by purge confirmation, or 3 to exit. Supply the repository root when prompted. Option 2 deletes only after the operator types the exact word PURGE.

Read-only dry scan

```
python3 /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py \
--repo '/mnt/YOUR_VOLUME/backups' --days 28 \
--report-dir '/var/log/veeam-xfs-cleanup' --dry-run
```

A dry scan inventories files and writes reports. It does not delete files. Supply an operator-writable private report directory when you do not run as root.

Confirmed scan and purge

```
sudo python3 /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py \
--repo '/mnt/YOUR_VOLUME/backups' --days 28 \
--report-dir '/var/log/veeam-xfs-cleanup' --scan-purge
```

Read the fresh groups_*.csv and manifest_*.json reports before typing PURGE. Any other answer cancels deletion. This mode requires an interactive terminal.

Unattended purge - no confirmation prompt

```
sudo python3 /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py \
--repo '/mnt/YOUR_VOLUME/backups' --days 28 \
--report-dir '/var/log/veeam-xfs-cleanup' --purge
```

Each destructive run performs a new inventory and eligibility check. It does not replay a previous CSV or manifest. Root and XFS are required.

Use --help for current switches. --days accepts 1 through 36500; the default is 28 elapsed days. The legacy --execute mode requires --confirm DELETE-EXPIRED-VEEAM-BACKUPS; prefer --scan-purge or --purge for new procedures.

4. Veeam v12 and v13 compatibility

The same program handles the two generations without a version flag. Existing v12 backup files can remain after upgrading the backup server to v13.

Example filename	Interpreted format
Example.vm-ABCD12342026-10-01T120000_ABCD.vbk	Legacy machine ID and direct timestamp
Example.vm-ABCD1234D2026-10-01T120000_ABCD.vib	Modern D-separated machine ID
ExampleD2026-10-01T120000.vbk	Documented name-only workload and timestamp
Example.vm-ABCD1234.vbm	Per-machine VBM metadata example

Recognized GUID-style identities and v12/v13 ID variants are normalized where the parser can prove their relationship. A newer incremental can therefore protect an older full, including after an upgrade. Unknown names, malformed IDs, unexpected extensions, and unfamiliar VBM XML

path references are held for review instead of deleted.

These patterns are not a promise to parse every Veeam workload. Reverse-incremental chains, unsupported metadata layouts, unresolved cross-directory references, and unreadable metadata are not automatically removed.

5. Retention and safe deletion decisions

The default threshold is 28 x 24 hours from execution. A complete independent VBK/VIB chain is considered only when every member has both an old recognized restore-point timestamp and an old file modification timestamp. A newer full in the same folder may start a separate chain. Any newer dependent incremental or retained VBM reference can protect the older full.

Decision / Meaning
PURGE_CANDIDATE / Eligible for another set of checks immediately before unlink
RETAIN / Required by a newer restore point or metadata dependency
IMMUTABLE / Protected by an immutable XFS file attribute
REVIEW / Ambiguous naming, shared or unreadable metadata, unsupported format, or other unresolved safety issue
FAILED_OR_BLOCKED / A proposed deletion stopped or failed; inspect the journal

The tool never modifies VBMs, removes .veeam.N.lock immutability tracking files, clears immutable attributes, updates GFS settings, changes VBR registration, or deletes directories. A .veeam.N.lock marker does not by itself prove that a Veeam job is running.

Before deleting candidate groups, the utility checks file identities, backup-file handles with fuser, metadata references, marker changes, and repository changes. It refuses nested or bind mounts, symlinks, incomplete files, and overlapping cleanup runs. These checks reduce risk but cannot prevent a Veeam writer from starting between checks.

6. Reports and audit trail

Reports default to ~/veeam-xfs-cleanup-reports for the invoking account, typically /root/veeam-xfs-cleanup-reports when sudo is used. Set --report-dir for unattended operation. Keep reports outside the repository.

Report pattern / Content
groups_*.csv / Chain-level decision, reason, counts, timestamps, and sizes
files_*.csv / File inventory and per-file decision
flagged_not_removed_*.csv / Older files left on disk, including dry-run candidates and protected or failed items
manifest_*.json / Informational scan manifest, never a subsequent purge input
deletions_*.jsonl / Purge-only durable journal with DELETE_INTENT, DELETED and failure records

After a destructive run, reconcile each DELETE_INTENT against DELETED and the actual files remaining. An intent without completion may indicate interruption; it does not prove the file survived. Logical bytes in the reports are not necessarily physically reclaimed blocks.

7. Scheduling

Schedule unattended purge only inside a guaranteed backup-copy idle window. For root crontab, edit with sudo crontab -e. The entry below must be a single physical line; the chosen time is

illustrative.

```
15 3 * * * /usr/bin/python3 /opt/veeam-xfs-cleanup/veeam_xfs_cleanup.py --repo '/mnt/YOUR
_VOLUME/backups' --days 28 --report-dir /var/log/veeam-xfs-cleanup --purge >> /var/log/ve
eam-xfs-cleanup/cron.log 2>&1
```

Monitor exit codes and retain the generated reports. A full-repository identity check happens before each eligible chain; large repositories can extend execution. Choose an idle window long enough for both scan and purge.

8. Exit codes

Code	Interpretation
0	Dry scan succeeded, or purge completed without flagged old files or failures
2	Refused before deletion: configuration, activity, non-XFS, unsafe path or lock
3	Purge interrupted or failed; partial deletion may already have happened
4	Purge completed but protected or otherwise flagged old files remain

An exit code of 0 from a dry scan does not mean that no purge candidates were found. Check the reports.

9. Troubleshooting and recovery

- Missing findmnt or fuser: install the relevant Linux package, then retry in the approved window.
- Non-XFS or nested mount refusal: verify the mount and repository scope. Do not bypass the check.
- Unreadable /proc/self/mountinfo: restore mount-info access before attempting purge.
- Files busy or repository changing: stop backup-copy jobs and wait for activity to cease.
- IMMUTABLE: allow the protection period to expire through the established backup policy; never force-clear flags.
- REVIEW: inspect the reason, machine-ID naming and VBM relationships; do not unlink individual VBK/VIB files.
- Exit 3 or unfinished DELETE_INTENT: disable scheduling, preserve all reports and reconcile on-disk state before rerunning.

To review recent console logs:

```
sudo ls -lt /var/log/veeam-xfs-cleanup
sudo tail -n 50 /var/log/veeam-xfs-cleanup/cron.log
df -h '/mnt/YOUR_VOLUME/backups'
```

10. Operator checklist

Before purge: verify backup-copy-only XFS root, Python/root privileges, findmnt/fuser, private report directory, and a guaranteed idle window. Confirm the 28-day cutoff. Review the candidate reports before any interactive confirmation.

After purge: retain groups, files, flagged, manifest, and deletion journal outputs. Investigate nonzero exit codes and any unfinished intent records. Compare free space separately. Confirm the next scheduled run will not overlap Veeam jobs.

11. Release and validation limits

v0.5.0 was approved for public GitHub source publication on October 13, 2026. The independent offline review included 146 regression cases and separate rubber-duck and debug passes on the same frozen cleanup code. This is evidence about synthetic fixtures and Linux test behavior, not Veeam vendor certification.

The release owner's previous manual v12 acceptance was for v0.4.3. Authentic v13 VBM samples and a native destructive test on mounted hardened XFS were not independently verified. Do not treat the GitHub release as authorization for unattended v13 production purge without environment-specific acceptance and a guaranteed idle window.

Source: https://github.com/eblackrps/xfs-file-cleanup